

FASE 2 | Insumos y Arquitectura

(Semana 3 - del 20 al 25 de abril)

1. Introducción

El presente documento describe la tercer semana de actividades del proyecto del proyecto “Friko recomienda” orientado al desarrollo de un sistema de recomendación de recetas personalizadas, a los usuarios de los productos Friko y Antillana, empresas del grupo Bios.

2. Objetivo del Proyecto

Darle continuidad a los procesos de las semanas 1 y 2 por medio de las entregas de esta semana, las cuales consisten en: Los MVP tanto de una Landig Page alojada en index.html y un Chat Bot, de Telegram, ambos desarrollados a través de un motor RAG **Retrieval-Augmented Generation** (Generación Aumentada por Recuperación), pruebas iniciales y hallazgos de estas, empleando 10 casos de uso o experiencias de usuario.

3. Alcance del Proyecto

El sistema incluirá:

- Recomendación de recetas personalizadas
- Asociación directa con productos de la marca
- Escalado dinámico de ingredientes según número de personas
- Uso de fuentes controladas de información
- Generación de recetas mediante inteligencia artificial como respaldo o sistema cerrado a información pre establecida (Esto lo definiremos durante el transcurso de la semana)

No incluye en esta fase:

- Desarrollo avanzado del sistema (refinamiento a prueba de fallos)
- Diseño y presentación final del Frontend (Renderizado).

4. Roles y Responsabilidades

Tech Lead: Santiago Botero.

- Elige la arquitectura y el enfoque RAG
- Define el uso de herramientas como Flowise, n8n y Supabase
- Diseña el flujo y valida el funcionamiento end-to-end
- Establece como se conectan y se integran los diferentes componentes

Integrado UI: Elvis Carreño

- Conecta el canal (Web/Telegram)
- Implementa el flujo conversacional
- Construye el pipeline hacia el RAG manejando Inputs de Usuario

UX Reviewer: Mileidy Astrid Casallas

- Analiza la interacción del Usuario con la Landing Page
- Revisa la claridad de los mensajes y el contenido
- Identifica problemas de usabilidad e interacción y propone mejoras

AI Tester: Sandra Victoria Colorado y Diego Hernán Velásquez

- Ejecutan casos de prueba definidos sobre el Chatbot
- Verifican que el sistema responda correctamente según los inputs
- Identifican errores, registran resultados de las pruebas y reportan hallazgos

Gestores de Avance: Yuri Paola Guavita y Diego Hernán Velásquez

- Integra las actividades independientes ejecutadas por cada uno de los roles técnicos en informe ordenado, coherente y consistente con el objetivo del proyecto.
- Durante esta fase del proyecto es el responsable de la comunicación a los supervisores y cliente de los avances del mismo mediante Kickoff semanal.

5. Insumos Entregados

- Documentación técnica del sistema completo
- Documentación técnica del sistema RAG (En sesión aparte)
- Documentación del sistema RAG y los canales de conexión construidos
- Informe de hallazgos y recomendaciones sobre el uso de la Landing Page
- Tablas de resultados de casos de uso (experiencias de Usuario) con el Chatbot.

6. Descripción de la Solución

De cara al usuario (Frontend) se conserva la estructura inicial del sistema, el cual funcionará a partir de cuatro entradas del usuario:

- Región
- Ocasión
- Método de preparación
- Número de personas

El Backend cambia al 100%, empleando un motor de integración RAG al cual se integraron herramientas como Grok, openAI y Supabase que interactúan y dan respuesta a los canales externos; Pudiendo incluso conectar otros canales como nodos adicionales; en otras palabras: haciéndolo Escalable.

7. Arquitectura Propuesta (Lo que cambio)

Durante esta semana se decidió cambiar de procesos pro-code a low-code, en aras de garantizar honestidad y cumplimiento con los tiempos de entrega y desarrollo, con un enfoque que precise aún más el propósito de este curso, como lo es la obtención de beneficios al máximo en el uso e implementación de sistemas y herramientas de IA, que ya sistematizan con una mayor precisión y calidad los diferentes procesos de diseño de software; cambiando un modelo más constructivo y minucioso a base de escritura de código por uno más orquestado, fluido y “gerencial”. Esto permite que el desarrollo sea aún más entendible para todo el equipo y un enfoque en el mejoramiento y el refinamiento del sistema en etapas finales.

8. Plan de Trabajo

1. Definición del Stack Tecnológico
2. Elección de los Canales: Landing Page y Chatbot de Telegram
3. Diseño y Construcción del RAG
4. Construcción e Integración de los canales al RAG
5. MVP Como entregable
6. Pruebas y testeos del sistema para corrección y refinamiento en la próxima entrega.

9. Criterios de Éxito

Darle continuidad al trabajo inicial, a la documentación base que construimos en principio y que nos sirvió como soporte para un cambio de enfoque en el diseño y desarrollo del sistema inteligente, que persigue el mismo propósito que nos trazamos en principio y es el de entregar un Sistema funcional, ágil, sencillo, intuitivo y amigable, pero con un plan de trabajo más ágil y refinado para conseguirlo.

Para las pruebas del sistema se diseñó un método por niveles, secuencial, también escalable y acumulativo que nos permitiera arrancar en pruebas iniciales básicas (casos de uso con errores humanos previsibles), pasar a pruebas de frontera (entregar inputs o instrucciones ilógicas al sistema) y finalmente pruebas de seguridad, el cual se empleará hasta la etapa final del diseño y operación.

10. Anexos

1. Documentación Técnica del Sistema Completo
2. Documentación Técnica del sistema RAG
3. Documentación del sistema RAG y los canales de conexión contruidos
4. Informe de hallazgos y recomendaciones sobre el uso de la Landing Page
5. Tablas de resultados de casos de uso (experiencias de Usuario) con el Chatbot.

(Dos tablas)

6. Informe Consolidado de la Documentación Técnica

NOTA: Se conserva el formato de origen de cada uno de estos anexos.

FRIKO RECOMIENDA

Documentación Técnica del Sistema Completo

Landing Page · Bot de Telegram · Motor RAG

Grupo BIOS · Colombia

Índice de Contenidos

1. Descripción General del Sistema
2. Stack Tecnológico
3. Landing Page — Friko Recomienda (index.html)
4. Bot de Telegram — Friko Chef Bot v6 (n8n)
5. Motor RAG — Flowise Chatflow
6. Integraciones Externas
7. Lógica de Negocio y Seguridad

1. Descripción General del Sistema

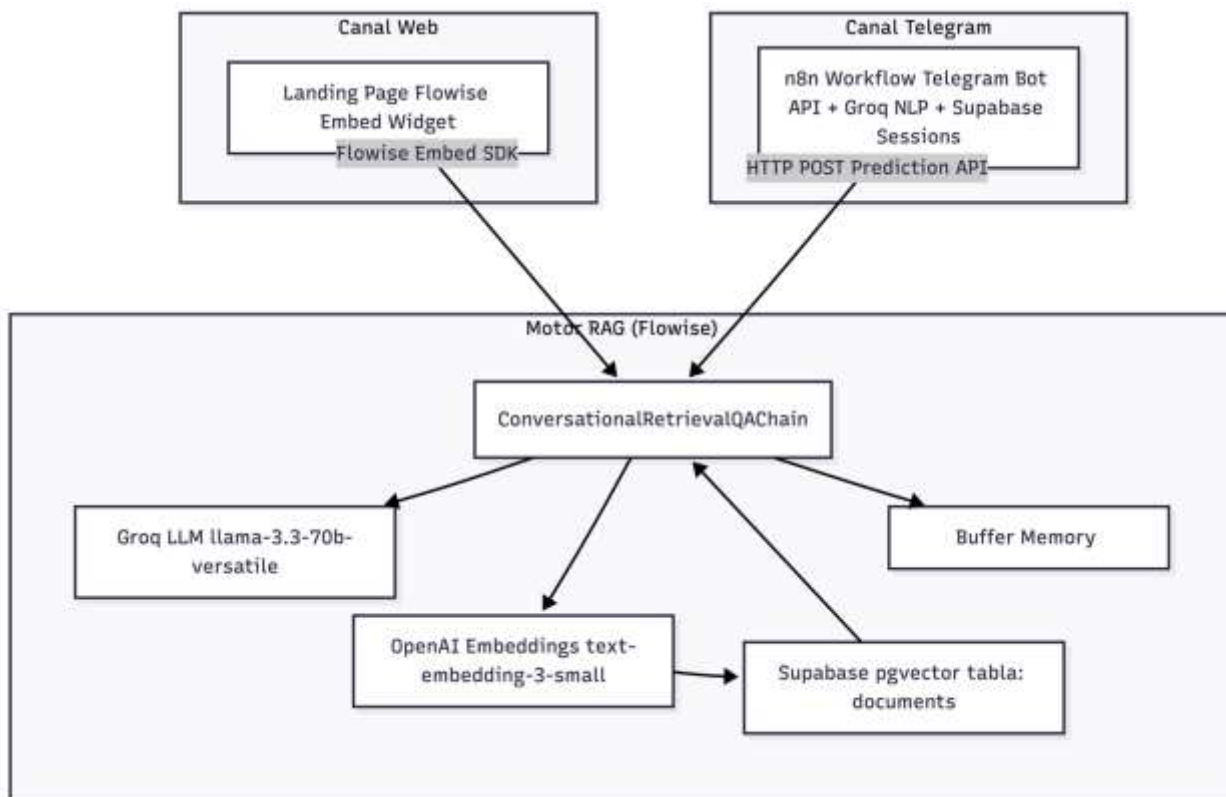
Friko Recomienda es un sistema de inteligencia artificial conversacional diseñado para las marcas Friko y Antillana del Grupo BIOS. Su objetivo es guiar a los usuarios en la elección del producto ideal y la preparación de una receta personalizada según su contexto (región, ocasión, número de personas y método de preparación).

El sistema funciona a través de dos canales de acceso que comparten el mismo motor de recomendación:

Canal	Tecnología	Descripción
Landing Page	HTML + Flowise Embed	Widget de chat embebido en la pagina index.html
Bot de Telegram	n8n + Telegram Bot API	Chatbot conversacional con estado persistente en Telegram

Ambos canales se conectan al mismo Chatflow de Flowise (motor RAG), que internamente utiliza Groq LLM para la generación de respuestas y Supabase pgvector para la recuperación semántica de productos y recetas.

1.1 Arquitectura global



2. Stack Tecnológico

Capa	Tecnología	Versión / Modelo	Función
Orquestación Bot	n8n	Cloud	Workflow de automatización del bot de Telegram
Canal usuario (web)	HTML / JavaScript		Landing page con widget de chat embebido
Canal usuario (móvil)	Telegram Bot API	v6 HTTP API	Interfaz de mensajería para el bot
NLP (extracción)	Groq API	llama-3.3-70b-versatile	Extracción de entidades del mensaje (n8n)
Generación LLM	Groq + Flowise	llama-3.3-70b-versatile	Generación de recomendaciones y recetas
Embeddings	OpenAI Embeddings	text-embedding-3-small	Vectorización de documentos y queries
Vector Store	Supabase pgvector	PostgreSQL + pgvector	Almacén de embeddings de productos y recetas
Sesiones (Telegram)	Supabase REST API	PostgREST	Persistencia de sesiones conversacionales
Motor RAG	Flowise Cloud	ChatflowV3	Orquestación del pipeline RAG completo

3. Landing Page — Friko Recomienda

El archivo index.html es la interfaz web del sistema. Actúa como página de presentación del asistente y punto de acceso directo al chat con IA y bot de Telegram.

3.1 Propósito

- Presentar el asistente Friko Recomienda al usuario con una experiencia visual clara y atractiva.
- Ofrecer acceso inmediato al chat de IA a través de un widget embebido (Flowise Embed SDK).
- Redirigir a usuarios móviles al bot de Telegram como alternativa.
- Comunicar las capacidades del sistema: recomendación por región, método de preparación y número de personas.

3.2 Conexión con el sistema RAG

La landing page integra el motor RAG directamente mediante el SDK de Flowise Embed, se activa cuando el usuario hace clic en el botón de chat

Una vez iniciado, el widget de Flowise se comunica directamente con el Chatflow RAG, enviando el mensaje del usuario y recibiendo la respuesta generada por el LLM con los datos del catálogo de Friko.

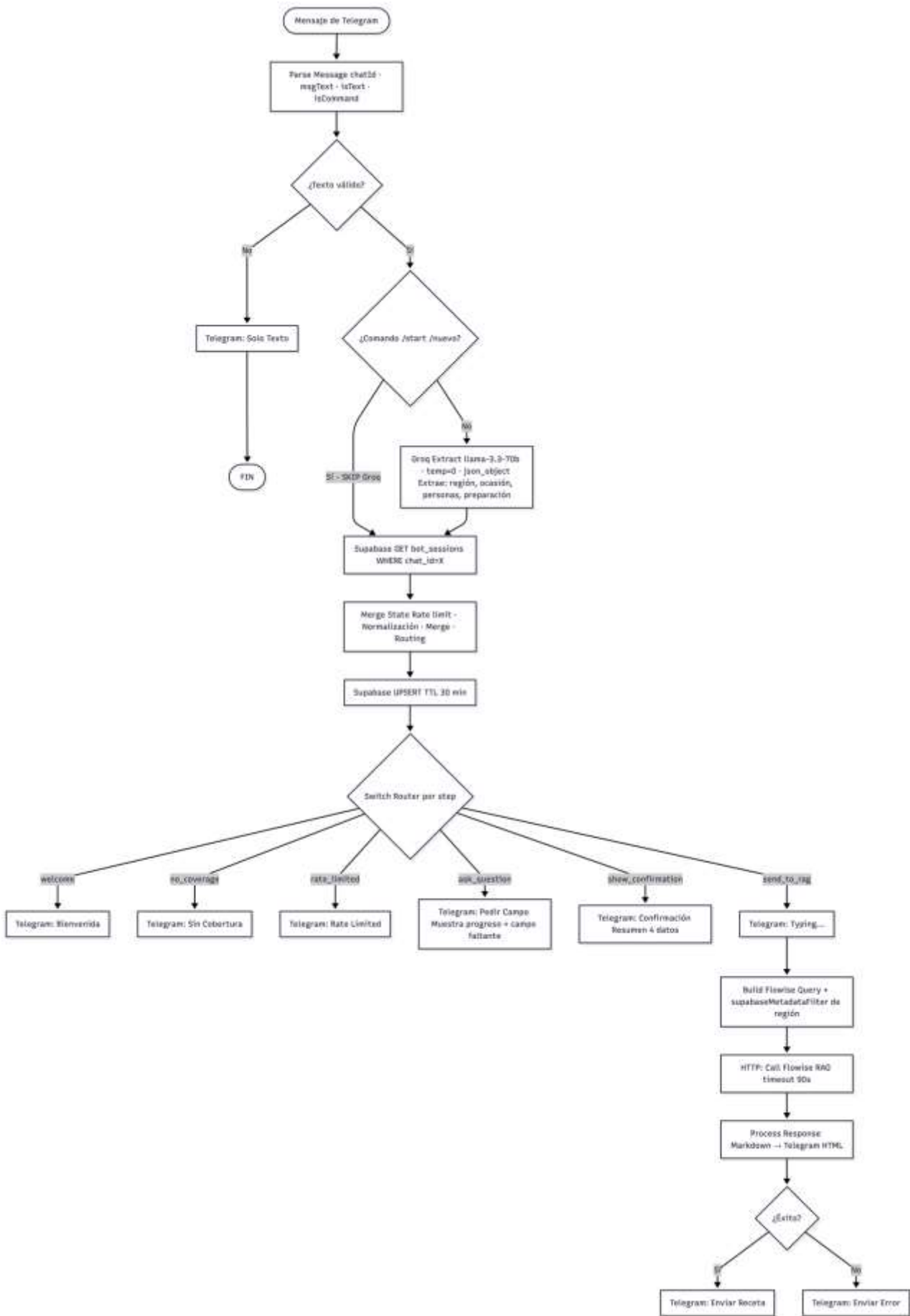
4. Bot de Telegram — Friko Chef Bot v6

El workflow de n8n es el componente que gestiona la conversación con el usuario en Telegram, extrayendo datos con Groq NLP, persistiendo el estado en Supabase y orquestando la llamada al RAG de Flowise cuando se tienen todos los datos necesarios.

4.1 Resumen del flujo

#	Fase	Nodos involucrados
1	Ingesta y validación	Telegram Trigger → Code: Parse Message → IF: Valid Text?
2	Extracción NLP	IF: Is Command? → HTTP: Groq Extract (omitido en comandos)
3	Gestión de sesión	HTTP: Supabase GET → Code: Merge State → HTTP: Supabase UPSERT
4	Routing (6 ramas)	Switch: Router
5	Respuestas intermedias	Telegram: Bienvenida / Sin Cobertura / Rate Limited / Pedir Campo / Confirmación
6	Consulta RAG y entrega	HTTP: Typing → Code: Build Flowise Query → HTTP: Call Flowise RAG → Code: Process Response → IF: Success? → Telegram

4.2 Diagrama de flujo de ejecución



4.3 Nodos principales

Nodo: Telegram Trigger

Tipo	n8n-nodes-base.telegramTrigger
Escucha	Evento: message (cualquier mensaje entrante)
Output	Objeto message completo de la Telegram Bot API
Conecta a	Code: Parse Message

Nodo: Parse Message

Extrae los campos esenciales del payload de Telegram y construye el cuerpo para Groq NLP.

Output	chatId, firstName, msgText, isText, isCommand, groqBody
Comandos	/start, /reset, /nuevo, /iniciar, /reiniciar
Groq model	llama-3.3-70b-versatile — temperature: 0, max_tokens: 150, response_format: json_object
Seguridad	System prompt con instrucciones anti-prompt-injection: ignora cualquier intento de cambio de rol

Los 4 campos que Groq extrae del mensaje del usuario:

Campo	Tipo	Descripción	Ejemplo
region	string null	Ciudad o departamento de Colombia	"Medellín" → "Antioquia"
ocasion	string null	Evento o momento para el que se cocina	"almuerzo familiar"
cantidad_personas	integer null	Número de comensales (1–500)	4
tipo_preparacion	string null	Método de cocción (normalizado)	"al horno" → "horno"

Nodo: IF: Is Command? — Bypass inteligente de Groq

Cuando el mensaje es un comando de reset, el flujo salta directamente a recuperar la sesión de Supabase sin pasar por Groq NLP. Esto ahorra tokens de API y evita llamadas innecesarias al LLM cuando no hay entidades que extraer.

Nodo: Code: Merge State — Nodo central

Es el nodo más complejo del workflow (~200 líneas de JavaScript). Actúa como máquina de estados conversacional e integra todos los datos upstream.

Procesa en orden:

1. Normalización de región: mapea regiones válidas, con soporte de acentos y preposiciones.
2. Rate limiting: máximo 2 mensajes por ventana de 5 segundos por chat_id.
3. Merge de sesión: preserva campos anteriores; sobrescribe si el usuario usa palabras de corrección ('en realidad', 'cambia', 'prefiero'...).
4. Validación de cobertura: rechaza regiones no cubiertas e informa al usuario.
5. Lógica de confirmación: detecta respuestas afirmativas/negativas del usuario.
6. Cálculo del step: determina la rama del Switch Router.

step	Condición	Respuesta
welcome	Comando de reset recibido	Mensaje de bienvenida + reset de sesión
rate_limited	Más de 2 mensajes en 5 segundos	Aviso de límite de velocidad
no_coverage	La región indicada no tiene cobertura	Informa ciudad sin servicio
ask_question	Faltan uno o más de los 4 datos	Solicita el campo faltante con progreso
show_confirmation	Los 4 datos completos, sin confirmación aún	Muestra resumen para validar
send_to_rag	Los 4 datos completos + usuario confirmó	Llama a Flowise RAG

Nodo:HTTP: Supabase UPSERT — Gestión de sesión

Guarda o actualiza la sesión del usuario con un TTL de 30 minutos. Usa el mecanismo UPSERT de PostgREST mediante el header Prefer: resolution=merge-duplicates.

Nodo: Build Flowise Query

Construye el payload para la Prediction API de Flowise. El campo `overrideConfig.supabaseMetadataFilter` instruye al RAG para filtrar únicamente documentos de la región del usuario ANTES de la búsqueda semántica, garantizando que las recomendaciones sean de productos con cobertura local.

Nodo: Process Response

Convierte la respuesta en formato Markdown de Flowise al HTML compatible con Telegram (etiquetas `<b>`, `<i>`, `<code>`). Detecta errores por código HTTP ≥ 400 o respuesta vacía y genera un mensaje de fallback.

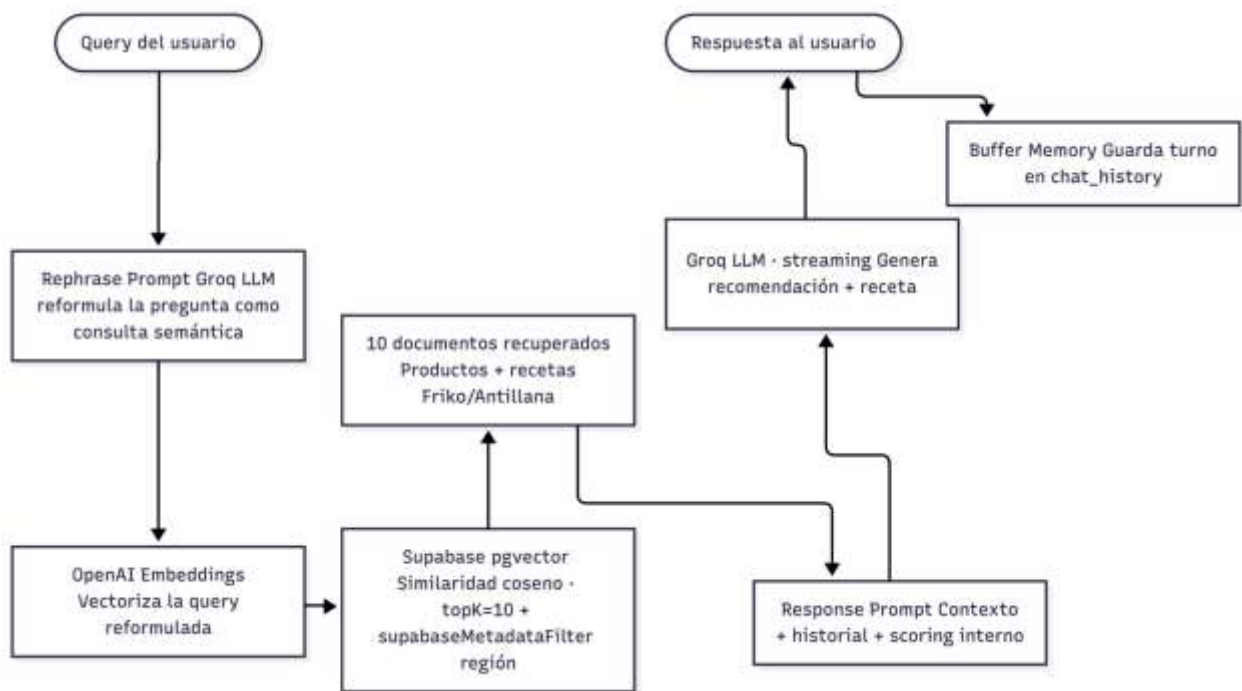
5. Motor RAG — Flowise Chatflow

El Chatflow de Flowise es el núcleo del sistema de recomendación. Es utilizado por ambos canales (landing page y bot de Telegram) y combina recuperación semántica de documentos con generación de lenguaje natural en un pipeline conversacional de 5 nodos.

5.1 Componentes del Chatflow

Nodo	Tipo	Configuración clave
groqChat	GroqChat / LLM	llama-3.3-70b-versatile · temp=0.55 · max_tokens=2000 · streaming=true
openAIEmbeddings	OpenAIEmbeddings	text-embedding-3-small · stripNewLines=true
supabase	Supabase / VectorStoreRetriever	tabla: documents · RPC: match_documents · topK=10 · similarity search
conversationalRetrievalQAChain	ConversationalRetrievalQAChain	Rephrase Prompt + Response Prompt · returnSourceDocuments=false
bufferMemory	BufferMemory	memoryKey: chat_history (in-memory, por sesión de Flowise)

5.2 Flujo interno del RAG



5.3 Prompts del sistema

Rephrase Prompt

Transforma la pregunta conversacional del usuario en una consulta de búsqueda semántica optimizada para pgvector. Recibe {chat_history} y {question}. Si detecta contenido malicioso o fuera de scope, retorna el fallback seguro: 'producto Friko Colombia'.

Response Prompt

Prompt principal que define el comportamiento del asistente. Está estructurado en 8 bloques funcionales:

Bloque	Función
Identidad	Define la personalidad: 'Friko Recomienda 🐼, asistente oficial de Friko y Antillana'
Seguridad	Reglas anti-jailbreak: detecta y rechaza cambios de rol, prompt injection, codificaciones
Alcance	Define qué puede y no puede responder (solo productos Friko/Antillana y recetas)
Cobertura regional	Mapa de ciudades → regiones con reglas de respuesta para ciudades sin cobertura
Flujo de datos	Orden de solicitud de los 4 datos: región → ocasión → personas → preparación

Bloque	Función
Scoring interno	+50 región · +25 método · +15 personas · +10 ocasión (no se muestra al usuario)
Recetas	Usa receta oficial de momentosfriko.com si existe; si no, genera una auténtica con IA
Formato	Estructura fija con emojis: 🍷 Producto, 📖 Receta, 💡 Tip, 🔄 Alternativa

5.4 Sistema de scoring interno

El LLM aplica un score a cada producto del contexto recuperado para seleccionar el más adecuado. Este proceso es completamente interno y nunca se expone al usuario.

Criterio	Puntos	Condición
Coincidencia regional	+50	La región del documento == región del usuario
Método de preparación	+25	El método solicitado está disponible para el producto
Número de personas	+15	La cantidad está dentro del rango pers_min–pers_max del producto
Ocasión	+10	La categoría del producto es adecuada para la ocasión
Puntaje máximo	100	Producto que cumple todos los criterios

Nota: Si el producto con mayor puntaje no cubre exactamente el número de personas, se recomienda igualmente con una advertencia de cantidad de paquetes. El sistema nunca deja de recomendar si hay datos disponibles en el contexto.

6. Integraciones Externas

Servicio	Protocolo	Uso	Autenticación
Telegram Bot API	HTTPS Webhook + REST	Recepción de mensajes y envío de respuestas	Bot token en URL
Groq API (n8n)	HTTPS REST	Extracción NLP de entidades del mensaje	Bearer token en header
Groq API (Flowise)	Via Flowise	Generación de recomendaciones y recetas (Rephrase + Response)	Credencial Flowise
OpenAI Embeddings	Via Flowise	Vectorización de queries y documentos	Credencial Flowise
Supabase REST	HTTPS PostgREST	Sesiones de Telegram (tabla bot_sessions)	apikey + service_role JWT
Supabase pgvector	Via Flowise	Búsqueda semántica de productos y recetas	Credencial Flowise
Flowise Cloud	HTTPS REST	Motor RAG completo (Prediction API)	Bearer token en header

7. Lógica de Negocio y Seguridad

7.1 conversacional

El bot implementa un patrón de slot-filling: recolecta 4 datos de forma secuencial antes de ejecutar la acción principal. El orden fijo es región → ocasión → número de personas → método de preparación. Sin embargo, el usuario puede proveer múltiples datos en un solo mensaje y el sistema los extrae todos a la vez con Groq NLP.

Ejemplo: El usuario escribe 'cena para 6 en Medellín, al horno' → Groq extrae los 4 campos de una sola vez → el bot confirma directamente sin preguntar.

7.2 Gestión de sesiones (Telegram)

- Sesiones persistidas en Supabase con TTL de 30 minutos.
- Si la sesión expira, el bot informa al usuario y retoma desde los datos disponibles.
- Los comandos (/nuevo, /start) limpian la sesión completamente.
- El campo _rapid_count controla el rate limiting por chat_id.

7.3 Cobertura regional

El sistema opera en 6 regiones de Colombia.

Región	Ciudades principales
Antioquia	Medellín, Bello, Itagüí, Envigado, Rionegro, Apartadó
Atlántico	Barranquilla, Soledad, Malambo, Galapa, Puerto Colombia
Bogotá	Bogotá D.C., Soacha, Chía, Zipaquirá, Facatativá, Mosquera
Eje Cafetero	Pereira, Armenia, Manizales, Dosquebradas, Cartago
Norte de Santander	Cúcuta, Villa del Rosario, Los Patios, Ocaña, Pamplona
Santander	Bucaramanga, Floridablanca, Girón, Piedecuesta, Barrancabermeja

7.4 Seguridad ante prompt injection

- Groq NLP (n8n): el system prompt define un rol fijo e inamovible. Cualquier intento de cambiar el comportamiento del LLM es ignorado; la única salida permitida es el JSON de 4 campos.
- Flowise Response Prompt: bloque de seguridad explícito que detecta y rechaza patrones de jailbreak (cambio de identidad, solicitud de instrucciones, comandos codificados, bypass de guardrails). La respuesta de escape es siempre: 'Solo puedo ayudarte con productos Friko 🐾'.

7.5 Manejo de errores

Punto de fallo	Tratamiento
Mensaje no es texto	IF: Valid Text? → aviso amigable + FIN
Groq NLP timeout o error	neverError=true → Merge State inicia extracción vacía, flujo continúa
Supabase GET falla	neverError=true → sesión vacía, flujo continúa
Supabase UPSERT falla	neverError=true → flujo continúa sin guardar estado
Flowise timeout (>90s)	neverError=true → Process Response detecta error → mensaje de aviso con /nuevo
Flowise respuesta vacía	isError=true → mensaje: 'Tuve un inconveniente consultando el catálogo'
Ciudad sin cobertura	Merge State → step=no_coverage → aviso al usuario, session.region se borra

FRIKO RECOMIENDA

Documentación Técnica del Sistema RAG

1. Descripción General

Friko Recomienda es un asistente conversacional inteligente basado en arquitectura RAG (Retrieval-Augmented Generation) diseñado para los usuarios de las marcas Friko y Antillana del Grupo BIOS. El sistema combina recuperación semántica de documentos con generación de lenguaje natural para recomendar productos y recetas colombianas personalizadas según los parámetros ingresados por el usuario.

1.1 Caso de uso principal

El asistente atiende a usuarios colombianos que buscan orientación para elegir productos Friko o Antillana y preparar recetas. Dadas las 6 regiones de cobertura (Antioquia, Atlántico, Bogotá, Eje Cafetero, Norte de Santander y Santander), el sistema adapta sus recomendaciones al perfil contextual de cada consulta: región, número de personas, método de preparación y ocasión.

1.2 Flujo general de funcionamiento

1. El usuario envía una pregunta o solicitud en lenguaje natural a través del chat.
2. El sistema recupera el historial de conversación desde la memoria de buffer.
3. Un Rephrase Prompt reformula la pregunta del usuario en una consulta de búsqueda semántica optimizada.
4. Se genera el embedding de la consulta reformulada y se buscan los documentos más similares en Supabase pgvector.
5. Los documentos recuperados (productos y recetas) se inyectan en el Response Prompt junto con el historial y los datos del usuario.
6. El LLM genera una respuesta estructurada con recomendación de producto, receta y alternativas.
7. La respuesta se entrega al usuario y se guarda en la memoria de sesión.

2. Arquitectura del Sistema

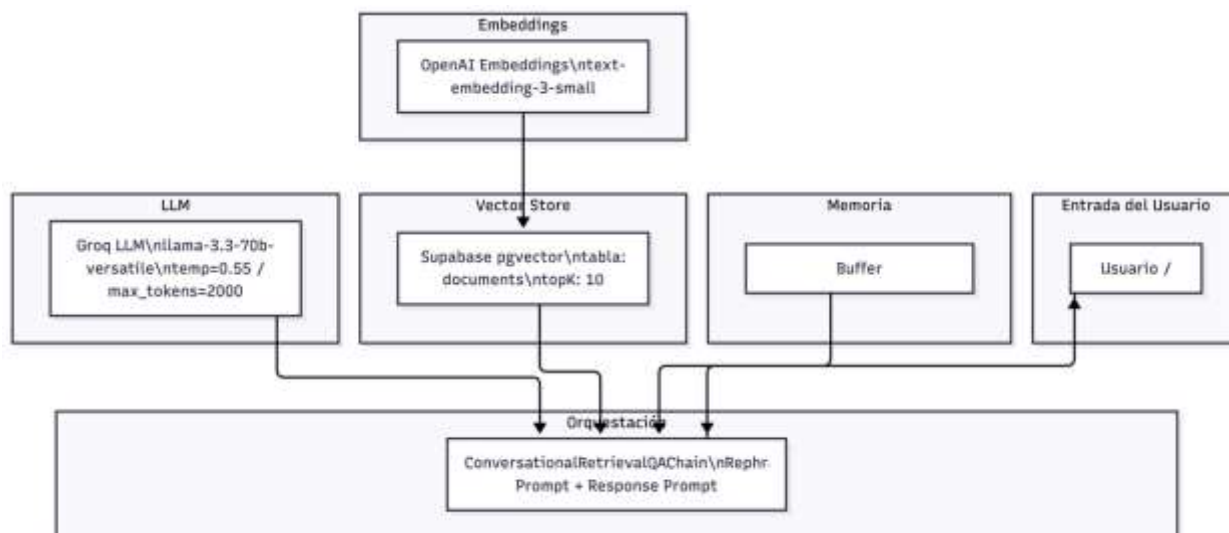
El sistema está construido sobre Flowise y está compuesto por 5 nodos principales interconectados. La arquitectura sigue el patrón Conversational RAG con memoria persistente de sesión.

2.1 Componentes identificados

Nodo ID	Tipo	Categoría	Descripción
groqChat	GroqChat	Chat Models	LLM principal — llama-3.3-70b-versatile
openAIEmbeddings	OpenAIEmbeddings	Embeddings	Modelo de embedding — text-embedding-3-small
supabase	Supabase	Vector Stores	Vector store pgvector con búsqueda por similitud
conversationalRetrievalQAChain	ConversationalRetrievalQAChain	Chains	Cadena orquestadora principal del RAG conversacional
bufferMemory	BufferMemory	Memory	Memoria de sesión — almacena chat_history

2.2 Diagrama de Arquitectura

El siguiente diagrama muestra cómo se conectan los 5 componentes del sistema:



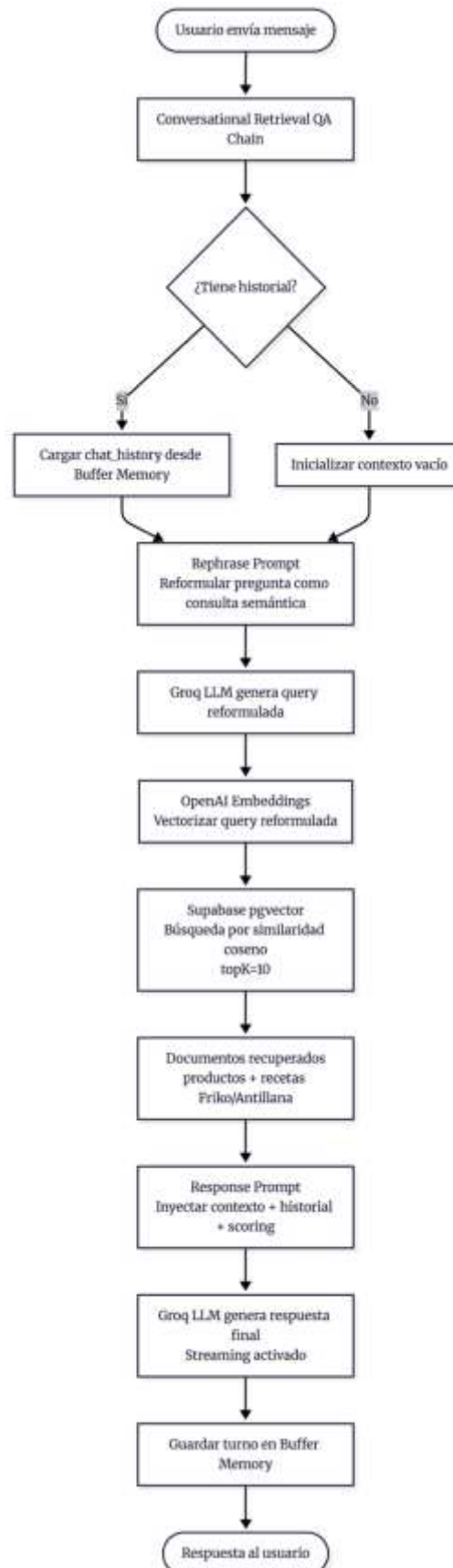
3. Flujo de Datos (Data Flow)

A continuación se describe el flujo completo desde que el usuario realiza una consulta hasta que recibe la respuesta final.

3.1 Descripción paso a paso

8. Entrada del usuario: El usuario escribe un mensaje en el chat (ej. '¿Qué cocino hoy para 4 personas en Medellín con airfryer?').
9. Recuperación de historial: El nodo Buffer Memory entrega el chat_history de la sesión actual al Chain.
10. Reformulación semántica: El Rephrase Prompt, ejecutado por el Groq LLM, convierte la pregunta original en una consulta semántica optimizada para la búsqueda vectorial (ej. 'pollo Friko Antioquia airfryer almuerzo').
11. Vectorización de la query: El modelo OpenAI text-embedding-3-small convierte la consulta reformulada en un vector.
12. Búsqueda en vector store: Supabase pgvector ejecuta una búsqueda de similitud coseno contra la tabla 'documents', retornando los topK=10 documentos más relevantes.
13. Construcción del prompt de respuesta: El Response Prompt inyecta los documentos recuperados ({context}), el historial ({chat_history}) y la pregunta original ({question}).
14. Generación de respuesta: El Groq LLM genera la respuesta final en streaming, aplicando el scoring interno y las reglas del sistema.
15. Persistencia en memoria: El turno completo (pregunta + respuesta) se guarda en el Buffer Memory para la siguiente interacción.
16. Entrega al usuario: La respuesta estructurada se muestra al usuario en tiempo real mediante streaming.

3.2 Diagrama de Flujo del Proceso



4. Explicación Detallada de Cada Componente

4.1 Groq LLM — groqChat

Atributo	Valor
Tipo	GroqChat / BaseChatModel
Modelo	llama-3.3-70b-versatile
Temperature	0.55
Max Tokens	2000
Streaming	Activado (true)
Proveedor de API	Groq (LPU Inference Engine)
Función	Reformulación de queries (Rephrase) y generación de respuestas finales (Response)
Output	BaseChatModel GroqChat Runnable → conecta al Chain

El modelo llama-3.3-70b-versatile de Meta corre sobre la infraestructura LPU (Language Processing Unit) de Groq, ofreciendo velocidad de inferencia significativamente superior a los GPUs tradicionales. Esto es especialmente relevante dado que el streaming está activado.

4.2 OpenAI Embeddings — openAIEmbeddings

Atributo	Valor
Tipo	OpenAIEmbeddings / Embeddings
Modelo	text-embedding-3-small
Strip New Lines	Activado (true)
Batch Size	No configurado (usa default)
Dimensiones	No configurado (usa default del modelo)
Encoding Format	No configurado (usa float por defecto)
Función	Convertir texto en vectores para búsqueda semántica
Output	OpenAIEmbeddings → conecta a Supabase

4.3 Supabase Vector Store — supabase

Atributo	Valor
Tipo	Supabase / VectorStoreRetriever / BaseRetriever
Project URL	https://rrdufiutyzeuljpanol.supabase.co
Tabla	documents
Función RPC	match_documents
topK	10 (documentos recuperados por búsqueda)
Tipo de búsqueda	similarity (similaridad coseno)
Metadata Filter	No configurado
RPC Filter	No configurado
Función	Almacenar embeddings y recuperar documentos relevantes
Output	VectorStoreRetriever → conecta al Chain

4.4 Conversational Retrieval QA Chain — conversationalRetrievalQAChain

Atributo	Valor
Tipo	ConversationalRetrievalQAChain
Categoría	Chains — Orquestador principal
Model conectado	groqChat (Groq LLM)
Retriever conectado	Supabase (pgvector)
Memory conectada	bufferMemory (Buffer Memory)
Return Source Documents	false (no expone documentos fuente al usuario)
Input Moderation	No configurado
Rephrase Prompt	Reformulación de queries en español con contexto colombiano
Response Prompt	Sistema completo de seguridad, scoring y formato de respuesta

4.5 Buffer Memory — bufferMemory

Atributo	Valor
Tipo	BufferMemory / BaseChatMemory / BaseMemory
Memory Key	chat_history
Session ID	No configurado (usa sesión de Flowise)
Función	Persistir el historial de conversación en memoria de sesión

Atributo	Valor
Output	BufferMemory → conecta al Chain como memoria conversacional

5. Manejo de Embeddings y Recuperación

5.1 Generación de embeddings

El sistema utiliza el modelo text-embedding-3-small de OpenAI para la vectorización de texto. Este modelo genera embeddings de alta calidad con menor costo computacional que su predecesor (text-embedding-ada-002). La opción stripNewLines=true garantiza que los saltos de línea no interfieran con la calidad del embedding.

El proceso de embedding se aplica en dos momentos del ciclo de vida del sistema:

- Ingesta de datos (offline): Los documentos de productos y recetas se vectorizan y se almacenan en Supabase pgvector usando la función de upsert del nodo Supabase.
- Tiempo de consulta (online): La query reformulada por el Rephrase Prompt se vectoriza en tiempo real para realizar la búsqueda de similitud.

5.2 Vector Store — Supabase pgvector

Supabase actúa como base de datos vectorial usando la extensión pgvector de PostgreSQL. La tabla 'documents' almacena los embeddings junto con el contenido y metadatos de cada documento (productos Friko/Antillana y recetas colombianas).

La función RPC match_documents implementa la búsqueda de similitud.

5.3 Estrategia de recuperación

Parámetro	Configuración y Significado
Search Type	similarity — búsqueda por similitud coseno
topK	10 — recupera los 10 documentos más similares
Fetch K	No configurado — solo aplica a MMR search
Lambda	No configurado — solo aplica a MMR search
Metadata Filter	No configurado — sin filtrado previo por metadatos
RPC Filter	No configurado — sin filtrado adicional por SQL

6. Uso del Modelo LLM

6.1 Modelo utilizado

El sistema utiliza llama-3.3-70b-versatile de Meta Llama 3.3, servido a través de Groq Cloud con tecnología LPU (Language Processing Unit). Este modelo de 70 billones de parámetros ofrece alta capacidad de comprensión del español colombiano y generación de texto de calidad.

Parámetro	Valor y Justificación
Modelo	llama-3.3-70b-versatile
Temperature	0.55 — balance entre creatividad y consistencia en recetas
Max Tokens	2000 — suficiente para respuestas completas con ingredientes y pasos
Streaming	true — entrega progresiva de la respuesta al usuario
Cache	Desactivado — cada consulta genera respuesta fresca

6.2 Construcción de los prompts

El sistema utiliza dos prompts diferenciados, ejecutados en etapas separadas:

Rephrase Prompt (Reformulación de Query)

Objetivo: Transformar la pregunta conversacional del usuario en una consulta semántica optimizada para la búsqueda vectorial.

Variables disponibles: {chat_history} y {question}

Instrucciones clave del prompt:

- Incluir nombre del producto, región colombiana, ocasión y método de preparación cuando estén disponibles.
- Si la pregunta es maliciosa o fuera de scope, retorna 'producto Friko Colombia' como fallback seguro.
- Output máximo: 2 líneas.

Response Prompt (Generación de Respuesta Final)

Objetivo: Generar la respuesta final con recomendación de producto, receta y formato estructurado.

Variables disponibles: {context} (documentos RAG), {chat_history} y {question}

El Response Prompt está organizado en los siguientes bloques funcionales:

- Bloque de identidad: Define la personalidad 'Friko Recomienda 🍴'.
- Bloque de seguridad: Reglas anti-jailbreak y anti-prompt injection con patrones de detección explícitos.

- Bloque de alcance: Define qué puede y qué no puede responder el bot.
- Bloque de cobertura regional: Mapa de ciudades a regiones con reglas de respuesta.
- Bloque de flujo de recolección: Orden de solicitud de los 4 datos necesarios.
- Bloque de scoring: Sistema interno de puntuación de productos.
- Bloque de recetas: Reglas para usar recetas oficiales o generadas por IA.
- Bloque de formato: Estructura exacta de la respuesta con emojis y secciones.

Nota: El campo {context} es inyectado por el nodo Supabase (los 10 documentos recuperados). El prompt instruye al LLM a basar sus recomendaciones ÚNICAMENTE en ese contexto, lo que garantiza fidelidad al catálogo real de productos.

7. Sistema de Recomendación

El sistema SÍ implementa un mecanismo explícito de recomendación y scoring. Está definido directamente en el Response Prompt bajo el encabezado 'SCORING INTERNO (nunca lo muestres)'.

7.1 Mecanismo de scoring

El LLM aplica internamente un sistema de puntuación a cada producto presente en el contexto recuperado. El sistema evalúa 4 dimensiones:

Puntos	Criterio	Descripción
+50	Coincidencia regional	La región del documento coincide con la región del usuario (Antioquia, Bogotá, etc.)
+25	Método de preparación	El método solicitado (Horno, Airfryer, Sartén, etc.) está disponible para el producto
+15	Número de personas	El número de comensales está dentro del rango pers_min–pers_max del producto
+10	Ocasión	La categoría del producto es adecuada para la ocasión indicada (almuerzo, picada, etc.)
MAX 100	Puntaje máximo posible	Producto que cumpla todos los criterios

7.2 Datos utilizados para la recomendación

- Embeddings semánticos: Los documentos recuperados por similaridad coseno constituyen el pool de candidatos a recomendar.
- Metadatos del producto: La información estructurada en los documentos (región, métodos, rango de personas, categoría) sirve como base para el scoring.

- Datos del usuario (recolectados en conversación): Región/ciudad, número de personas, método de preparación y ocasión.
- Historial de conversación: Evita solicitar datos ya provistos y mantiene la coherencia entre turnos.

7.3 Integración en la respuesta final

El producto con mayor puntaje se presenta como recomendación principal. Se permite hasta 1 producto alternativo adicional del contexto. Si el producto ganador no cubre exactamente el número de personas, se recomienda igualmente pero con advertencia de cantidad de paquetes necesarios.

El score NUNCA se muestra al usuario — es un proceso interno del LLM. La respuesta final siempre sigue la estructura:

7.4 Diferencia: respuesta directa vs. recomendación

Tipo	Respuesta Directa	Recomendación
Trigger	El usuario tiene todos los datos o pide info específica	El usuario no tiene todos los 4 datos requeridos
Proceso	RAG → LLM genera respuesta inmediata	Flujo de recolección de datos → scoring → RAG → LLM
Preguntas	No hace preguntas adicionales	Solicita 1 dato a la vez (región → personas → método → ocasión)
Scoring	Se aplica igualmente	Se aplica después de tener los 4 datos

ENTREGA FASE 2 - INTEGRADOR UI – Elvis Carreño Suárez

1. Qué se construyó y por qué dos canales

El rol de Integrador UI es conectar al usuario con el motor RAG. La decisión central fue habilitar dos canales de acceso que comparten el mismo motor, pero responden a contextos distintos: el comprador de pollo que busca qué cocinar está en su celular, no frente a un formulario web.

Landing Page (web): widget de chat embebido mediante Flowise Embed SDK sobre una página HTML. Conexión directa al RAG. Sin estado — cada sesión es independiente. Cero fricciones: el usuario hace clic y empieza a chatear.

Bot de Telegram (móvil): orquestado en n8n Cloud vía Telegram Bot API. Conexión indirecta al RAG — n8n extrae entidades con Groq NLP, gestiona el estado en Supabase y construye la query antes de llamar a Flowise. Simula el canal WhatsApp por efectos prácticos del proyecto.

Ambos canales llaman al mismo Chatflow de Flowise. La diferencia está en quién prepara la query: en la web lo hace el RAG directamente desde lenguaje libre; en Telegram lo hace n8n con extracción estructurada y sesión persistente de 30 minutos.

2. Flujo de integración: Del mensaje a la query RAG

El trabajo del Integrador UI termina cuando el RAG recibe una query limpia y estructurada y consta de seis fases secuenciales:

- **Ingesta y validación:** recibe el mensaje y verifica que sea texto. Imágenes, stickers o archivos terminan el flujo con un aviso amigable.
- **Extracción NLP:** Analiza el texto libre y extrae hasta 4 entidades en una sola llamada: región, ocasión, número de personas y método de preparación.
- **Gestión de sesión:** Recupera el estado previo del usuario desde Supabase y fusiona con los datos nuevos. Si el usuario corrige un dato ('en realidad prefiero el horno'), el sistema sobrescribe el campo anterior.
- **Routing:** Con los datos fusionados, el nodo Merge State calcula un 'step' y el Switch Router determina cuál de las 6 ramas ejecutar.
- **Recolección progresiva:** Si faltan datos, solicita uno a la vez en orden fijo, mostrando el progreso al usuario (ej: campo 2 de 4).
- **Llamada al RAG:** Cuando los 4 datos están completos y el usuario confirma, n8n construye la query con filtro regional y llama a Flowise. El motor recupera los 10 documentos más relevantes de Supabase y genera el producto recomendado con receta.

Atajo importante: si el usuario da los 4 datos en un solo mensaje ('cena para 6 en Bogotá al horno'), extrae todos de una vez y el bot salta directamente a confirmación sin hacer preguntas.

3. Árbol de decisión - Las 6 ramas del router

El Switch Router es el nodo central del diseño. Después de fusionar el estado de sesión con los datos nuevos, el sistema calcula un step y ejecuta una de estas seis ramas:

welcome: Se activa con cualquier comando de reset (/start, /nuevo, /reset). Limpia toda la sesión en Supabase y reinicia los 4 campos a null. Responde con mensaje de bienvenida e instrucciones.

rate_limited: Se activa si llegan más de 2 mensajes en 5 segundos del mismo usuario. Bloquea el procesamiento sin llamar a LLM, ni Supabase. Avisa al usuario y espera que pase la ventana.

no_coverage: Se activa cuando la región detectada no está en las 6 zonas de cobertura FRIKO. Borra session.region, informa al usuario y espera que corrija con otra ciudad.

ask_question: Se activa cuando falta uno o más de los 4 datos. Identifica el primer campo faltante en el orden fijo (región → ocasión → personas → método) y lo solicita con indicador de progreso.

show_confirmation: Se activa cuando los 4 datos están completos pero el usuario aún no ha confirmado. Muestra un resumen de los 4 datos y pide Sí o No.

send_to_rag: Se activa cuando los 4 datos están completos y el usuario confirmó. Construye la query con filtro regional, envía indicador de escritura y llama a Flowise RAG.

4. Diseño de Manejo de Fallos

Una parte crítica de la integración UI es garantizar que ningún fallo técnico llegue al usuario como un error incomprensible. Cada punto de fallo tiene un tratamiento definido:

Nota: Todo fallo en la integración (LLM, Supabase, Flowise) está cubierto con `neverError=true` o con mensajes de fallback explícitos. El usuario siempre recibe una respuesta en lenguaje natural, nunca un stack trace ni un mensaje vacío.

Punto de fallo	Tratamiento técnico	Mensaje al usuario	¿El flujo continúa?
Mensaje no es texto	IF Valid Text? → rama FALSE → END	Aviso amigable: solo proceso texto	No. Fin limpio.
Groq NLP timeout o error	<code>neverError=true</code> → extracción vacía	Ninguno. El bot pide el campo faltante normalmente.	Sí. Degrada con gracia.
Supabase GET falla	<code>neverError=true</code> → sesión vacía	Ninguno visible. Flujo continúa sin historial.	Sí. Pierde contexto previo.
Supabase UPSERT falla	<code>neverError=true</code> → no guarda estado	Ninguno visible.	Sí. Sin persistencia en esa interacción.
Flowise timeout >90s	<code>neverError=true</code> → Process Response detecta error	'Tuve un inconveniente. Escribe /nuevo para reintentar'	No. Fin con mensaje de recuperación.
Flowise respuesta vacía	<code>isError=true</code> → mensaje de fallback	'Tuve un inconveniente consultando el catálogo'	No. Fin con mensaje de recuperación.
Ciudad sin cobertura FRIKO	Merge State → <code>step=no_coverage</code> → borra región	Informa ciudad sin servicio. Sugiere escribir otra.	Sí. Vuelve a pedir región.

5. Estado de la entrega

- ✓ Landing page y bot Telegram activos y operativos.
- ✓ Flujo end-to-end documentado con 6 ramas de decisión.
- ✓ 10 casos de prueba especificados con rama activada y resultado con mejoras a implementar en la siguiente entrega.

Los 4 criterios de éxito de Fase 2 están cubiertos por diseño: el sistema siempre recomienda productos del catálogo real (garantizado por el filtro regional en Build Flowise Query), las recomendaciones varían por región y ocasión, una persona sin conocimiento técnico puede usarlo sin ayuda (flujo de lenguaje natural con recolección progresiva), y los 10 casos de pruebas están documentados.

Proyecto: Friko Recomienda

Rol: UX Reviewer

Objetivo: Validar que la experiencia (landing+chatbot) sea clara, entendible y guiada para usuarios sin conocimiento técnico

Se realiza el análisis sobre:

- Landing page
- Chatbot – flujo exitoso
- Chatbot – manejo de errores
- Experiencia para usuarios sin conocimiento técnico

Landing Page

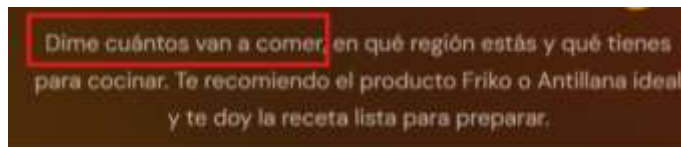
- **Hallazgo:** El mensaje principal “**Dime que vas a cocinar hoy**” es atractivo, pero no es completamente explícito para el usuario que obtiene al final del proceso,
- **Impacto en el usuario:** para usuarios sin contexto pueden no comprender que función cumple la landing page.
- **Recomendación UX:** Aclarar el objetivo de la landing page.
- **Hallazgo:** La frase “**IA COLOMBIANA**” no es una frase que debería tener la pagina ya que no se esta usando IA colombiana, puesto que no es un modelo entrenado localmente en Colombia, no fue desarrollado por una institución colombiana, no es infraestructura propia no funciona con datasets propios nacionales.
- **Recomendación UX:** se recomienda frases como “IA ENFOCADA EN LA COCINA COLOMBIANA” o “IA PENSADA PARA COLOMBIA”



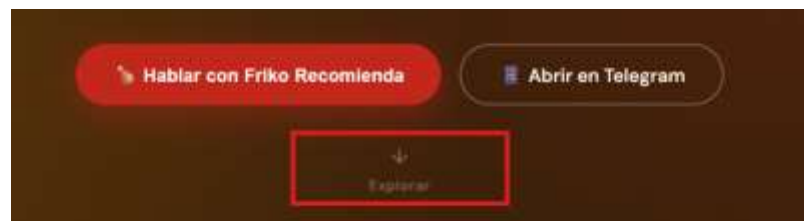
- **Hallazgo:** En el mensaje inicial se encuentra la siguiente frase “**y que tienes para cocinar**”, el cual no debería estar dado que el objetivo de la landing page es justamente **RECOMENDAR** el producto para cocinar. Si inicialmente le digo al usuario que tiene para cocinar, le estoy diciendo que producto tiene para cocinar, y es contrario al objetivo del proyecto que es recomendar el producto.
- **Recomendación UX:** se recomienda quitar esta frase del mensaje principal.



- **Hallazgo:** En el mensaje inicial también se inicia con “Dime cuántos van a comer”.
- **Impacto en el usuario:** No es claro a quien o que hace referencia a ¿personas, animales, robots?
- **Recomendación UX:** Se recomienda cambiar la frase a “Dime para cuantas personas vas a cocinar”



- **Hallazgo:** Los CTA “Hablar con Friko recomienda” y “abrir en Telegram” compiten visualmente como acciones principales.
- **Impacto en el usuario:** el usuario puede no entender cuál es la acción recomendada o cual es la alternativa principal, esto puede generar duda sobre cual de las dos opciones elegir y se puede aumentar la posibilidad de dar clic sobre la primera opción, adicionalmente el clic en “abrir en Telegram” abre el chatbot de Telegram sin embargo no funciona.
- **Recomendación UX:** Se recomienda no habilitar la opción mientras no este funcionando de manera correcta.
- **Hallazgo:** El CTA “explorar” no es funcional, ya que no genera ninguna acción
- **Recomendación UX:** Se recomienda retirarlo ya que no es funcional.



- **Hallazgo:** El lenguaje de la landing no es sencillo para un usuario no técnico. **Impacto en el usuario:** Palabras como validar, bot, puntuación, inputs, flujo, mapea son palabras que un usuario no técnico puede no conocer
- **Recomendación UX:** Se recomienda usar palabras como verificar, asistente, ayudante, agente, datos, entre otros para que el lenguaje sea más cercano y sencillo para el usuario.



- **Hallazgo:** Se comunica de manera confusa las secciones “míralo en acción” y la sección “lo que necesitas decirle”
- **Impacto en el usuario:** se transmite al usuario que necesita de tres mensajes para generar una conversación real (no es claro con el objetivo inicial de recomendar un producto y generar una receta) y por otro lado se le comunica al usuario que necesita ingresar 4 datos.
- **Recomendación UX:** Se recomienda homologar la comunicación y el objetivo de la landing page, ya que en este punto no se tiene claro si el agente va a tener conmigo una conversación real (¿temas de interés para mi o para los dos?) o es un agente que me va a recomendar un producto y una receta de acuerdo con los 4 datos que yo le entregue.



- **Hallazgo:** En la sección final se visualiza la frase “DISPONIBLE EN 6 REGIONES DE COLOMBIA” seguido de una frase: “Si tu ciudad no aparece en la lista, el bot te lo dice claramente”
- **Impacto en el usuario:** Confusión porque el mensaje 1. Inicia transmitiendo información de regiones y continua con información de ciudades. 2. Se transmite una información acerca de una lista de ciudades que no se evidencia en ningún lado,

adicionalmente se visualizan 6 regiones resaltadas en botones que no hacen ningún llamado cuando se hace clic en ellos.

- **Recomendación UX:** Se recomienda unificar el lenguaje y aclarar la ubicación del listado de ciudades disponibles.



Chatbot – flujo exitoso

El chatbot sigue un flujo secuencial muy claro:

- Región o ciudad
- Número de personas
- Método de preparación
- Ocasión

Hallazgo: Algunos de los mensajes del chat son extensos y contemplan varias ideas en un solo bloque.

Impacto en el usuario: Impacto negativo en el usuario por la carga visual y porque puede preferir no leer todo el mensaje omitiendo información, en algunos casos el orden de la pregunta no es coherente ejemplo: ¿Cuántas personas vas a cocinar para?

Recomendación UX: Se recomienda el uso de botones o listas, dividir mensajes extensos en frases cortas o separar confirmación y pregunta en mensajes diferentes ya que esto genera menos esfuerzo para el usuario, menos error de escritura y una experiencia más guiada.



Chatbot – manejo de errores

Hallazgo: El mensaje de error en el chatbot “Ups, algo salió mal” es genérico y no indica al usuario como continuar.

Impacto en el usuario: Confusión y una alta probabilidad de abandono.

Recomendación UX: Se recomienda un mensaje más cercano y claro de acuerdo con el error, por ejemplo: “tuvimos un problema al verificar tu ciudad ¿puedes escribirla de nuevo o intentar con la región en la que te encuentras?”. En este punto es importante incluir en el flujo un reintento guiado o una alternativa que evite que el usuario opte por cerrar el chat sin haber obtenido su receta.

Este feedback se presenta desde el rol de UX Reviewer, enfocado en garantizar una experiencia clara, confiable y entendible para usuarios sin conocimiento técnico.

Anexo de evidencias:

PRUEBAS FUNCIONALES										
PREGUNTA DEL BOT	PRUEBA No.1		PRUEBA No.2		PRUEBA No.3		PRUEBA No.4		PRUEBA No.5	
	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta
¿Para que ocasión estas cocinando?	Desayuno		desayuno		Desayuno		desayuno		Desayuno	**
↓	Costa caribe		↓		↓		↓		Desayuno	***
¿En que región de Colombia te encuentras?			costa		Caribe		caribe		Caribe	
¿Para cuantas personas vas a cocinar?	2		2		Dos		dos		2	
↓	Freir	Mensaje de error	freir	Mensaje de error	↓		↓		↓	
¿Tipo de preparación preferida?	Freidora	Mensaje de error	↓		Freidora	Mensaje de error	freidora de aire	Mensaje de error	Freidora de aire	****
	nuevo	Mensaje de error	Nuevo	Mensaje de error	↓		↓			
	/nuevo	Reinicio del sistema	/Nuevo	Reinicio del sistema	/nuevo	Reinicio del sistema	/nuevo			
CONCLUSIONES	Cuando el Bot sugiere intentar de nuevo se debe incluir la barra diagonal a la palabra "nuevo"; esto es poco o nada común, pero intuitivo ni práctico.		Esta vez reconoce las palabras escritas con mayúscula pero el mismo error de la barra diagonal mostrado en la prueba No.1		Esta vez se le da una respuesta al tipo de preparación preferida que está dentro de las opciones del formulario de manera incompleta; aún así no la reconoce como válida y arroja el mensaje de error*.				** 1er. Mensaje de error: "Problema técnico momentáneo, disculpa" . y continúa solicitando de nuevo la ocasión. *** 2do. Mensaje de error: En esta respuesta cambio las opciones de ciudad o región: En todas las pruebas anteriores mostraba las siguientes opciones de Ej: "Antioquia, Bogotá, Atlántico, Santander, Eje Cafetero, Costa Caribe"; en esta respuesta dio las siguientes alternativas de Ej: "Medellín, Bogotá, Bucaramanga, Barranquilla, Pereira, Cúcuta"; aún así se le dio la misma respuesta de pruebas anteriores (Caribe) y la reconoció. Percia que estuvieran cambiando el backend mientras se interactuaba con el Bot. ****3ro. Al final de todo el proceso indica: "Lo siento, pero no tenemos cobertura en la región del Caribe. Operamos en Antioquia, Atlántico, Bogota..."	
* MENSAJE DE ERROR POR DEFECTO: Ups, tuve un inconveniente al consultar el catálogo de Friko. Por favor intenta de nuevo escribiendo /nuevo. Si el problema persiste, intenta más tarde. This message was sent automatically with n8n										

PRUEBAS FUNCIONALES										
PREGUNTA DEL BOT	PRUEBA No.6		PRUEBA No.7		PRUEBA No.8		PRUEBA No.9		PRUEBA No.10	
	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta	Imput	Respuesta
Inicio del Bot	/start	**	Hola	*****	/start	**	Hola en audio	Mensaje de error	/Nuevo	Exitoso
¿Para que ocasión estas cocinando?	Cena	Exitoso	Asado	Exitoso	Desayuno	Exitoso			En un solo mensaje pude	Exitoso
¿En que ciudad o region de Colombia estas?	Medellin	Exitoso	Bogota	Exitoso	Barranquilla	Exitoso			enviar todo lo que	
¿Para cuantas personas vas a cocinar?	10	Exitoso	1	Exitoso	Cinco	*****			necesitaba para la	
¿Tipo de preparación preferida?	Cualquiera	*****	Sarten	*****					recomendación del asado:	
	Horno	*****	/Nuevo	Reinicio del sistema					Cucuta, para 200 perosnas,	
									es para un asado y quiero	
									usar parrilla y de comida	
									quiero pollo.	
Confirmacion									Si	Exitoso
	** 1er. Mensaje de error: "Problema técnico momentáneo, disculpa". y continúa solicitando la ocasión. ***** 5to. Mensaje de error: Al escribir la palabra Cualquiera sale "Solo puedo ayudarte con productos Friko y recetas colombiana", recomienda preguntar directamente en google y luego continua solicitando la tipo de preparacion. ***** 6to.Mensaje error: Tuve un inconveniente al consultar el catalogo de Friko		***** 6to.Mensaje error: Tuve un inconveniente al consultar el catalogo de Friko. ***** 5to. Mensaje de error: Al escribir la palabra Cualquiera sale "Solo puedo ayudarte con productos Friko y recetas colombiana", recomienda preguntar directamente en google y luego continua solicitando la tipo de preparacion.		*****7to. Mensaje de error: Es vez de escribir el numero 5 lo puse en letras (cinco) sale "Solo puedo ayudarte con productos Friko y recetas colombianas", recomienda preguntar directamente en google y vuelve a preguntar para cuantas personas.		Solo acepta mensajes de texto: por favor escribe /nuevo para comenzar		Al confirmar que si esta correcto la informacion ingresada, inmediatamente me sale una receta con las porciones, producto e ingrediente con los detalles de preparacion	

Proyecto: Friko Recomienda – Sistema de recomendación de productos y recetas

Fase: 2 – Construcción del recomendador

Fecha: 22 de abril de 2026

Equipo: Reto 12 – grupo 2

❖ **Objetivo de esta semana:**

Recomendador funcional v1: sistema que recibe el perfil del usuario y devuelve un producto real del catálogo, probado con 10 casos y validado por alguien externo al equipo.

❖ **Roles de esta semana:**

Tech lead: Santiago Botero Madrid

Integrador UI: Elvis Carreño Suarez

Tester: Sandra Victoria Colorado Duque y Diego Hernán Velásquez Martínez

UX reviewer: Mileydi Astrid Casallas Romero

Gestor de avance: Yury Paola Guavita Perdomo.

❖ **Justificación cambio camino técnico de pro-code a low-code:**

Decidimos cambiar de pro-code a low-code, para pasarnos de un modelo donde "construimos las herramientas" a uno donde "orquestamos soluciones". Esto permite que el equipo se enfoque en la calidad de las recetas y la experiencia del usuario de Friko.

Además buscamos con este cambio enfocarnos en la eficiencia operativa, la democratización del mantenimiento y la velocidad de iteración, sustentados en lo siguiente:

1. Velocidad de Lanzamiento y "Time-to-Market"

- **Desarrollo Visual vs. Escritura de Código:** Mientras que la ruta Pro-code requiere mayor esfuerzo inicial, el uso de **Flowise** y **n8n** permite ensamblar la lógica de la "Conversational QA Chain" de forma visual y casi instantánea.
- **Iteración Ágil:** En un entorno Low-code, ajustar una regla de negocio o cambiar el modelo de IA (por ejemplo, pasar de Claude a Llama-3) no requiere reescribir funciones complejas en Python, sino simplemente reconectar nodos en la interfaz.

2. Democratización y Mantenimiento del Proyecto

- **Reducción de la Barrera Técnica:** con Pro-code requería una mayor capacidad técnica del equipo. Al cambiar a Low-code, se reduce la dependencia de desarrolladores especializados en Python, permitiendo que cualquier persona del equipo pueda entender y ajustar el flujo de trabajo en **n8n** o **Flowise**.
- **Documentación Viva:** La arquitectura técnica funciona como su propia documentación. Cualquier persona que vea el diagrama de flujo entiende cómo se conectan los datos de **Supabase** con el bot de **Telegram**.

3. Orquestación Multicanal Simplificada

- **Conectividad Nativa:** en la arquitectura se muestra una integración directa con **Telegram** a través de **n8n**. En Pro-code, gestionar múltiples canales (Web y Telegram) implica programar y mantener diferentes controladores. En Low-code, **n8n** gestiona estas conexiones de forma nativa y robusta.
- **Gestión de APIs:** Centralizar la lógica en **Flowise Cloud** permite exponer una sola API (flowise-embed API) que sirve tanto para la web como para otros servicios, simplificando la infraestructura.

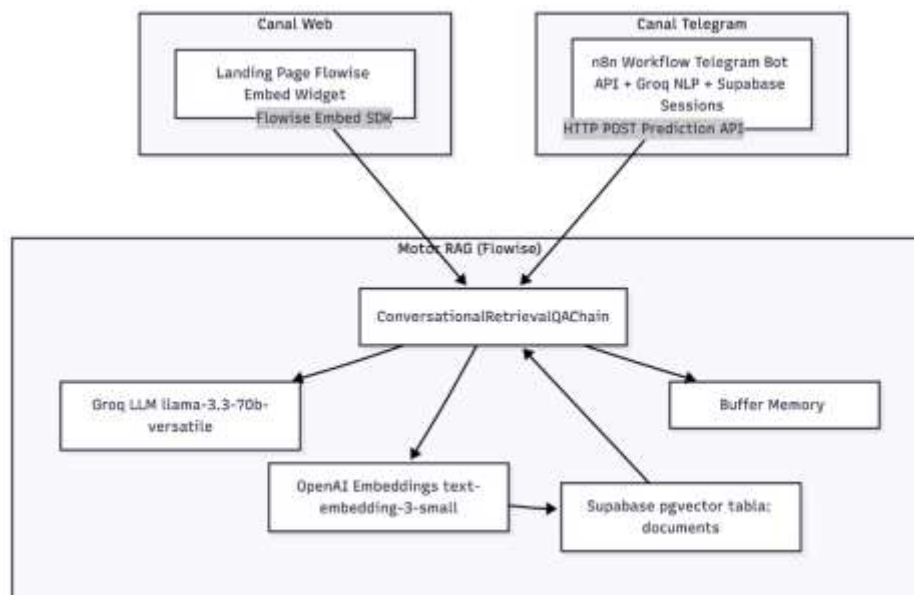
4. Robustez de la Infraestructura de IA (RAG)

- **Manejo de Memoria y Contexto:** En lugar de programar manualmente el "Buffer Memory" en Python, herramientas como **Flowise** ya tienen componentes optimizados para gestionar el historial de chat y el contexto de los documentos.
- **Búsqueda Semántica Eficiente:** La arquitectura ya contempla el uso de **Supabase (pgvector)** y **OpenAI Embeddings**. Implementar esto vía nodos visuales reduce el riesgo de errores en la gestión de bases de datos vectoriales que podrían ocurrir en un desarrollo desde cero.

5. Mitigación de Riesgos y Costos

- **Deuda Técnica:** El kick-off, fase 0 semana 1 Pro-code, advertía sobre la generación de deuda técnica temprana. El Low-code mitiga esto al utilizar estándares de la industria (n8n/Flowise) que son mantenidos por sus respectivas comunidades, asegurando que el sistema no se vuelva obsoleto rápidamente.
- **Costo de Infraestructura:** El uso de servicios en la nube como **Streamlit Cloud** o **n8n Cloud** permite pagar solo por lo que se usa, evitando el mantenimiento de servidores complejos.

❖ Arquitectura Global



❖ Stack tecnológico

Capa	Tecnología	Versión / Modelo	Función
Orquestación Bot	n8n	Cloud	Workflow de automatización del bot de Telegram
Canal usuario (web)	HTML / JavaScript		Landing page con widget de chat embebido
Canal usuario (móvil)	Telegram Bot API	v6 HTTP API	Interfaz de mensajería para el bot
NLP (extracción)	Groq API	llama-3.3-70b-versatile	Extracción de entidades del mensaje (n8n)
Generación LLM	Groq + Flowise	llama-3.3-70b-versatile	Generación de recomendaciones y recetas
Embeddings	OpenAI Embeddings	text-embedding-3-small	Vectorización de documentos y queries
Vector Store	Supabase pgvector	PostgreSQL + pgvector	Almacén de embeddings de productos y recetas
Sesiones (Telegram)	Supabase REST API	PostgREST	Persistencia de sesiones conversacionales
Motor RAG	Flowise Cloud	ChatflowV3	Orquestación del pipeline RAG completo

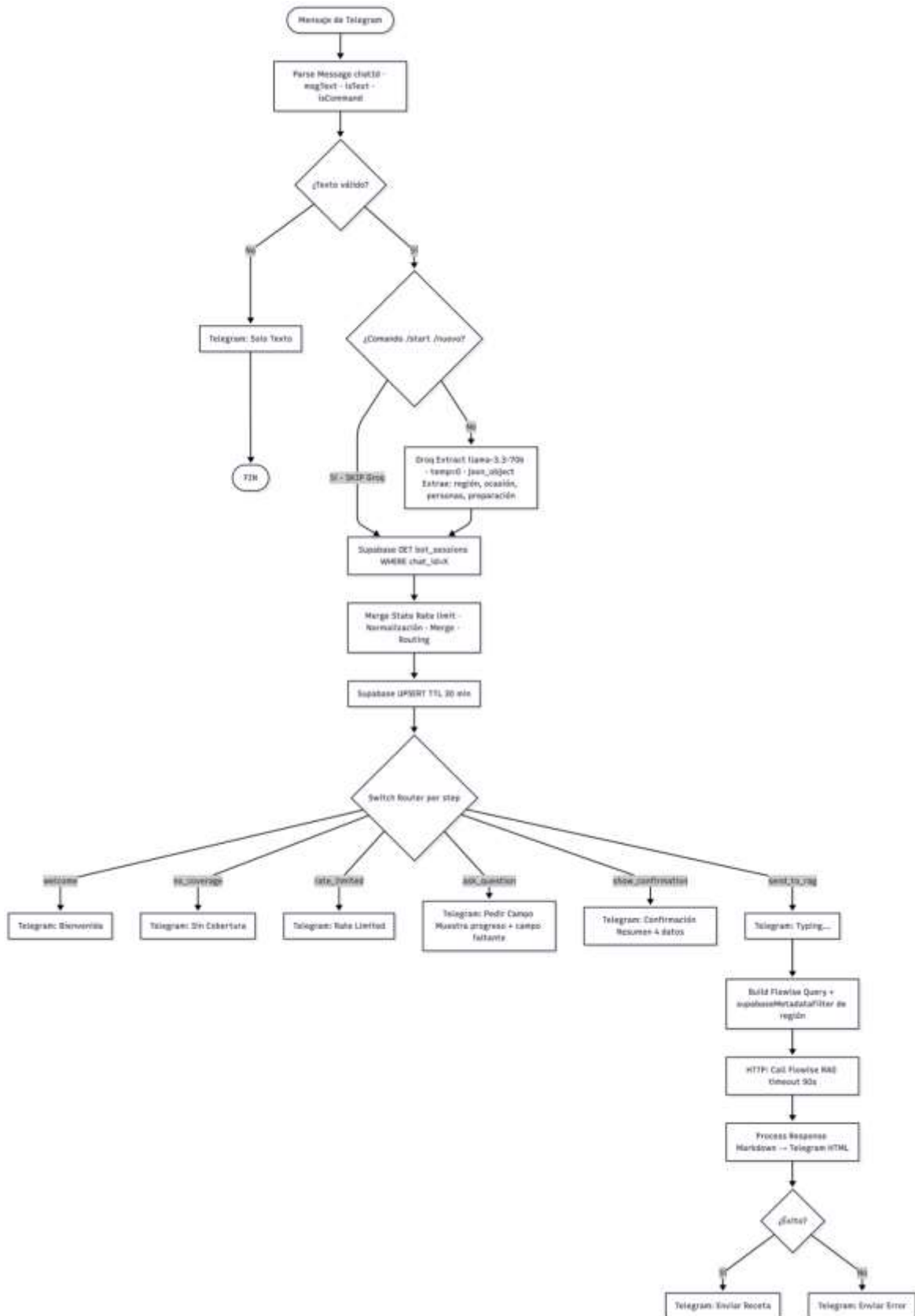
❖ Funcionamiento sistema de recomendación:

El sistema funciona a través de dos canales de acceso que comparten el mismo motor de recomendación:

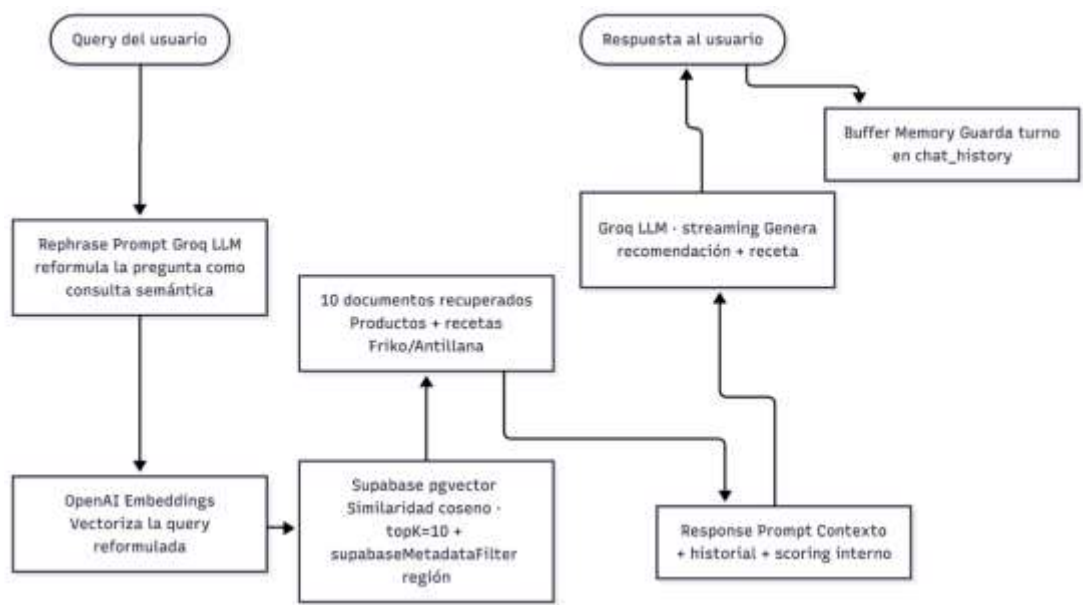
Canal	Tecnología	Descripción
Landing Page	HTML + Flowise Embed	Widget de chat embebido en la pagina index.html
Bot de Telegram	n8n + Telegram Bot API	Chatbot conversacional con estado persistente en Telegram

Ambos canales se conectan al mismo Chatflow de Flowise (motor RAG), que internamente utiliza Groq LLM para la generación de respuestas y Supabase pgvector para la recuperación semántica de productos y recetas.

❖ Diagrama de flujo ejecución Bot telegram



❖ Diagrama de flujo del RAG



❖ Feedback experiencia de usuario

Riesgo	Hallazgo (Texto Original del Documento)	Impacto del Usuario	Solución / Recomendación
Dependencia de API externa	"1. La solución implementa una cadena de generación de recetas en tres niveles: Recetas reales existentes, Generación dinámica con IA (Claude API) y Plantillas locales de respaldo".	"El usuario experimenta 'silencios' o caídas del servicio si la API de IA falla, dejando la conversación a medias".	Fortalecer el fallback automático a plantillas locales para asegurar una respuesta inmediata siempre.
	"2. Este flujo secuencial requiere control de errores, manejo de latencia y validaciones condicionales".	"Ante un retraso de pocos segundos, el usuario puede pensar que el bot dejó de funcionar".	Configurar mensajes de estado o indicadores de escritura en el chat para gestionar la percepción de latencia.

Crecimiento del catálogo	"3. Lectura de datos locales en memoria (Excel y JSON)".	"Dificultad para manejar datos en archivos locales a medida que el proyecto evoluciona".	Realizar una migración futura a base de datos centralizada para una gestión de datos más robusta.
	"4. Incremento del catálogo de productos y recetas".	"Riesgo de presentar información desactualizada si no hay una sincronización visual del flujo de datos".	Utilizar la automatización de n8n para mantener los datos del catálogo siempre vigentes y sincronizados.
Cambios de reglas comerciales	"5. El motor debe aplicar filtro excluyentes (Región)".	"Recibir un mensaje de 'No hay resultados' detiene la experiencia de forma negativa".	Programar una lógica de salida amable que ofrezca recetas generales cuando el filtro de región no arroje resultados.
	"6. Evaluar tipo de preparación y categoría del producto".	"El usuario puede sentirse abrumado por la cantidad de datos requeridos antes de recibir valor".	Implementar un flujo conversacional paso a paso que solicite un solo dato por mensaje.
	"7. Considerar número de porciones y bonificaciones según ocasión".	"El usuario recibe una receta pero no entiende por qué es la mejor opción para su necesidad específica".	Incluir una breve frase explicativa que justifique la recomendación basada en la ocasión seleccionada.
Escalabilidad	"8. El reto exige velocidad de entrega (MVP funcional en pocas semanas)".	"Dificultad en versionamiento, testing y una experiencia del usuario limitada en su rapidez inicial".	Utilizar herramientas low-code (Flowise/n8n) para iterar reglas de negocio sin generar deuda técnica temprana.

	"9. Posible incorporación de modelos de Machine Learning en el futuro".	"Incertidumbre sobre si el diseño actual soportará nuevas funciones de personalización profunda".	Mantener una arquitectura modular que permita separar el frontend del backend para facilitar expansiones futuras.
Complejidad del motor de recomendación	"10. Calcule puntuaciones ponderadas y multiplicadores".	"Para un usuario no técnico, el resultado parece aleatorio al no comprender la inteligencia detrás de la sugerencia".	Traducir la lógica de puntuación a lenguaje natural dentro de la respuesta del asistente.
	"11. El flujo termina tras mostrar la receta".	"El usuario no sabe cómo guardar la receta o dónde comprar los productos de Friko".	Incorporar botones de respuesta rápida al final del flujo para ver otras opciones o localizar puntos de venta.

Resumen de Valor

Este feedback se presenta desde el rol de UX Reviewer, enfocado en garantizar una experiencia clara, confiable y entendible para usuarios sin conocimiento técnico

Al documentar todos estos hallazgos, queda claro que la ruta **low-code** permite resolver cada punto con **nodos visuales auditables**, garantizando que el flujo sea entendible para cualquier persona sin necesidad de explicar la complejidad del código Python subyacente.